

Feed Forward Neural Networks & Back-propagation

C K Lekhana
24.01.2026

Keywords Feed-Forward Neural Networks, Back-propagation, Regression Loss Function

1 Motivation

My journey to so far with Deep Learning & Computer Vision was mostly restricted to using models off the shelf from different libraries or borrowing from publicly available code implementations of different architectures. While I had a surface level understanding, I often failed to explain why a model is built this way, why this works better than the other or justify a model architecture with deep understanding, even to myself. So, I decided to learn/build the intuition of what actually is a neural network. This article exactly describes my through process while trying to understand the foundational concepts (Neuron, Feed Forward Neural Networks & Back-propagation) of DL/ Neural-Networks.

2 Neural Networks

From a zoomed out perspective, Neural Networks can just be describe in the following way.

Neural Networks are interconnections of “layers” , which are further made up of parallelly executing “neurons”.

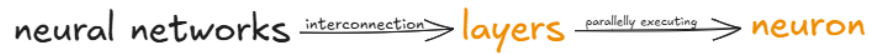


Figure 1: Hierarchy of Neural Networks Components

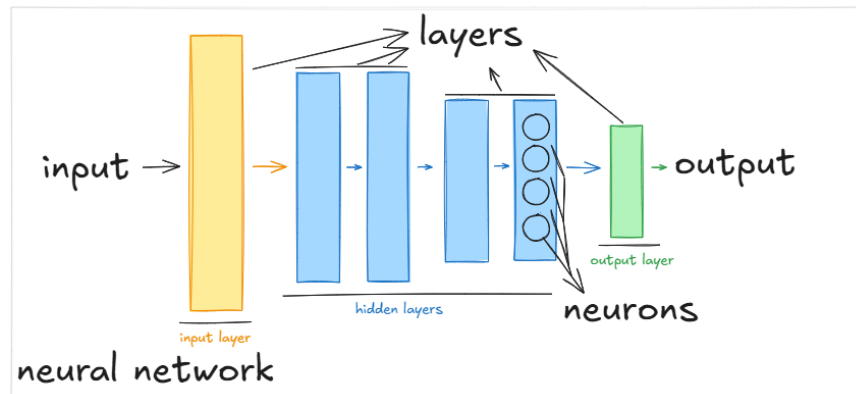


Figure 2: A Simple Neural Network

A neural network has three different layers as seen in fig.1 & 2. The input layer takes each of the features from input and passes it to all the neurons in the following layer. Hidden layers are responsible for actually performing the transformation of input that simplifies the prediction process. Output layer is the layer responsible for producing the final results for the input passed through the network.

Now, that we understand the building blocks of the neural networks. We can look at the following questions:

1. What does a single neuron do?
2. What happens during forward pass / Forward-propagation within a neural network?
3. How can we train a Feed-forward neural network without Back-propagation?
4. Why is Back-propagation necessary?
5. What happens during backward pass / Back-propagation within a neural network?
6. How are the weights updated?

[Note: All the questions above will be answered with a regression task in mind.]

3 What does a Single Neuron do?

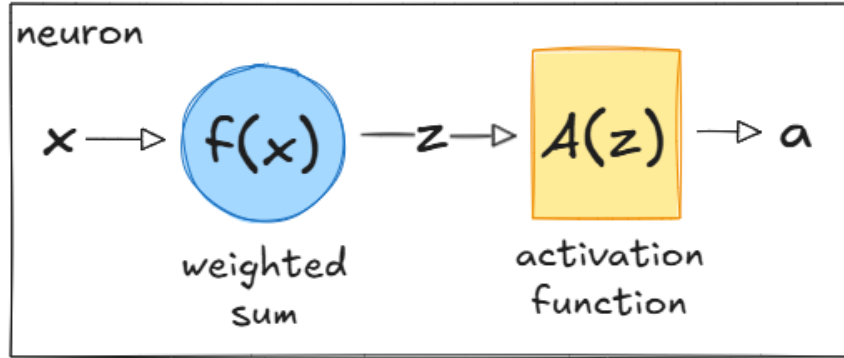


Figure 3: A Single Neuron and its components.

A neuron can be defined as a mathematical model comprising of two functions: Weighted Sum function & Activation function.

Weighted Sum Function The Weighted Sum function $f(x)$ takes a set of inputs x_i , set of weights w_i and bias b to output a real-value z . For a N -featured input x , $f(x)$ is given as,

$$z = f(x) = \sum_{i=1}^N x_i * w_i + b \quad (1)$$

The above equation is for a single input of features. In reality, we will have data of size (M, N) , where M is the number of data points/ rows in the dataset and N is the number

of features/ columns in the dataset. For a given j^{th} sample in the dataset, then z_j is given as,

$$z_j = \sum_{i=1}^N x_{ij} * w_i + b \quad (2)$$

Derivatives w.r.t weights w_j , bias b and input x_{ij} is given as,

$$\frac{dz_j}{dw_j} = x_{ij} \quad (3)$$

$$\frac{dz_j}{db} = 1 \quad (4)$$

$$\frac{dz_j}{dx_{ij}} = w_i \quad (5)$$

Activation Function The activation function A_i is the key component in the neuron/ neural network. It is responsible for transforming the input data into a different form which helps in predicting the output/ target variable better. We have different type of activation functions. Following are the main ones:

Linear This activation performs no operation on the input z_j and just returns z_j . It is majorly used at the output layer for regression tasks.

$$a_j = z_j \quad (6)$$

$$\frac{da_j}{dz_j} = 1 \quad (7)$$

Sigmoid This activation is takes a real-value input z_j and maps it to a value between 0 & 1 to output a_j . It is given as:

$$a_j = \frac{1}{1 + e^{-z_j}} \quad (8)$$

$$\frac{da_j}{dz_j} = a_j * (1 - a_j) \quad (9)$$

tanh

$$a_j = \frac{1 - e^{-z_j}}{1 + e^{-z_j}} \quad (10)$$

$$\frac{da_j}{dz_j} = 1 - a_j^2 \quad (11)$$

ReLu

$$a_j = \max(0, z_j) \quad (12)$$

$$(13)$$

These neurons execute parallelly within a layer and share the same input x_i and activation function $A(z)$. Each of the neurons within the layer will have different weights and bias, required for calculating the weighted sum.

4 Background

Before, we dive into understanding the workings of the feed forward neural network. We have to understand the following notations used later in the article.

For any given regression task, we have the following:

$X \Rightarrow$ Input matrix of size (M, N)

$y \Rightarrow$ Output matrix of size $(M, 1)$

In this discussion, we will exclude the input layer since it is just a set of N neurons that allows passing of exactly 1 feature within it. This layer has no weights or biases attached, so it is safe to exclude it. In general, a layer l_i has the following components to it.

Output of the weighted sum in $l_i \Rightarrow z_i$

Output of the activation function of $l_i \Rightarrow a_i$

Input to $l_i \Rightarrow$ Output of the activation function of $l_{i-1} \Rightarrow a_{i-1}$ or X (in case of the first hidden layer)

Weights of each neuron n_k in $l_i \Rightarrow w_i$

Bias of each neuron n_k in $l_i \Rightarrow b_i$

The shape of w_i and b_i can generally be written as (N_{i-1}, N_i) and $(1, N_i)$. Here, N_{i-1} is the number of neurons in the previous layer/ the number of features/ columns outputted by the previous layer. For l_1 , N_{i-1} is the number of features in the input matrix X . N_i is the number of neurons in l_i or the number of features outputted by the l_i . Here, each column represents the weights and bias of neuron. The shape of z_i and a_i depends on the shapes of other components in l_i .

For example, if the layer has 5 neurons, then the layer requires 5 set of weights and 5 biases to perform computation. So, w_i and b_i will have 5 columns for each of the neurons. The number of rows in w_i depends on the number of neurons in the previous layer or the number of columns in the output of the previous layer.

$$\begin{aligned}
 X_{(M \times N)} &= \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1N} \\ x_{21} & & & \\ \vdots & & & \\ x_{M1} & & & x_{MN} \end{bmatrix} & y_{(M,1)} &= \begin{bmatrix} y_{11} \\ y_{21} \\ \vdots \\ y_{M1} \end{bmatrix} \\
 w_i_{(N_{i-1}, N_i)} &= \begin{bmatrix} w_{1n_1} & w_{2n_2} & \dots & w_{N_{i-1}n_1} \\ \vdots & & & \\ w_{N_{i-1}n_1} & & & \end{bmatrix} & b_i_{(1, N_i)} &= \begin{bmatrix} b_{1n_1} & b_{2n_2} & \dots & b_{N_{i-1}n_1} \end{bmatrix}
 \end{aligned}$$

Figure 4: Layer Components as matrices.

All the components discussed are in the form of matrix as shown in fig. 4.

5 What happens during forward pass/ forward propagation within a neural network?

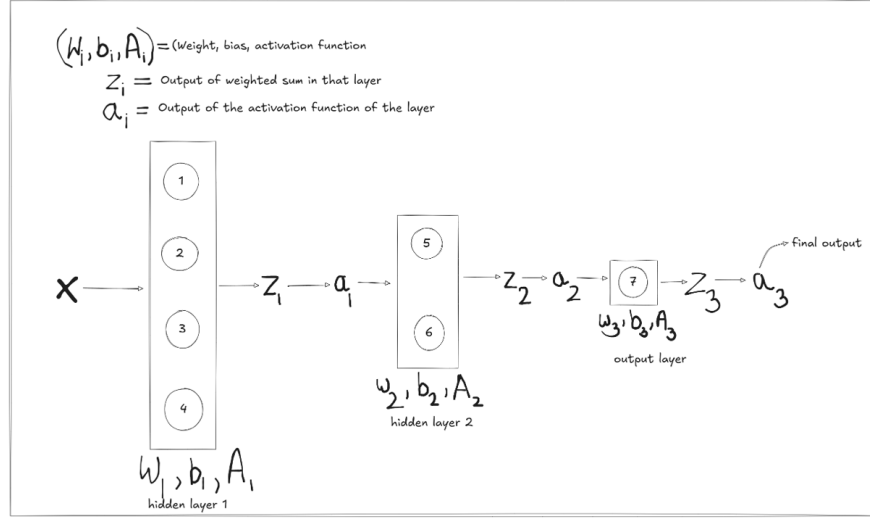


Figure 5: Forward Propagation in a Simple Neural Network

$$\begin{matrix} x \\ (M, N) \end{matrix} \Rightarrow \begin{matrix} a_1 \\ (M, 4) \end{matrix} \Rightarrow \begin{matrix} a_2 \\ (M, 2) \end{matrix} \Rightarrow \begin{matrix} a_3 \\ (M, 3) \end{matrix}$$

During forward propagation, each layer just transforms the input into output based on the activation function. The computation here is similar to what happens at the neuron level as discussed earlier. The only difference being that of scale. Each neuron within the layer will now parallelly perform the computation to get the output. Since, we are considering all the components of layer as vectors/ matrices. We don't have to perform calculations for each neuron, as its computations are already accounted for, at the layer components (discussed earlier) it belongs to.

For a simple neural network, we can abstract all the calculations like in fig. 5. Each layer has three defining characteristics, that are, w_i , b_i and A_i . The activation function A_i remains the same throughout the model life unless explicitly changed for valid reasons. During the training process, w_i and b_i change accordingly to accommodate the data. The shapes of the outputs of each layer changes as follows. We can observe that the number of features after each layer changes as per the number of neurons in the layer, as in fig. .

6 How can we train a feed forward neural network without backpropagation?

The method to train the Feed Forward Neural Network is to use the gradient descent method. But, it is important to note that we can only change the weights of output layer in this method. Because, without back propagation, it is not possible to find the change in loss due to change in weights in the hidden layers. So, we assume that the hidden layers already have the correct weights and update/ modify the weights of the output layer through gradient descent.

To train via gradient descent, we need to define the Loss function first. The loss function L helps us to determine the loss in predicting the target for a given input sample. In regression, the popular choice is Mean Squared Error. Given as:

$$L = \frac{1}{2} \sum_{i=1}^M (y' - y)^2 \quad (14)$$

So, in the case of a neural network, this loss is written as:

$$L = \frac{1}{2} \sum_{i=1}^M (a_{k=lastLayer} - y)^2 \quad (15)$$

Then the gradient w.r.t to $a_{k=lastLayer}$ of this is given as:

$$\frac{dL}{da_{k=lastLayer}} = \sum_{i=1}^M (a_k - y) \quad (16)$$

Since, we need to update the weights and bias of the output layer, we need to determine the gradient of L wrt $w_{k=lastLayer}$ and $b_{k=lastLayer}$, respectively.

$$\frac{dL}{dw_{k=lastLayer}} = \frac{dL}{da_k} \frac{da_k}{dz_k} \frac{dz_k}{dw_k} \quad (17)$$

From Eq. 3 & 16,

$$\frac{dL}{dw_{k=lastLayer}} = \sum_{i=1}^M (a_k - y) \cdot \frac{da_k}{dz_k} \cdot a_{k-1} \quad (18)$$

$$\frac{dL}{db_{k=lastLayer}} = \frac{dL}{da_k} \frac{da_k}{dz_k} \frac{dz_k}{db_k} \quad (19)$$

From Eq. 4 & 16,

$$\frac{dL}{db_{k=lastLayer}} = \sum_{i=1}^M (a_k - y) \cdot \frac{da_k}{dw_k} \cdot 1 \quad (20)$$

From above we get the gradients for the weight and bias of output layer, which can be used to update the weights and biases. This way we can train the neural network, this process is similar to that in the case of training a linear regression via gradient descent.

7 Why Back Propagation is necessary?

Clearly, from the previous question, we can see that the training is based on the assumption that the weights & bias for $l_{k \in \text{hiddenLayer}}$ is ideal and correct. This beats the very objective of a neural network, which is to optimally transform the input feature space to another feature space for efficient target prediction. Therefore, it is important to determine a method that helps us to update these weights and bias belonging to the hidden layers. Back-propagation is one such method that helps us to determine the optimal weights for a given dataset.

8 What happens during backward pass/ back propagation within a neural network?

The idea of back propagation is very simple. For each of the weights & biases in the model, we need to identify how much it contributes to the final output. This is done by simply propagating the loss backwards. This [foundational paper](#) introduces back-propagation as finding the gradient/ derivative of the loss L w.r.t w_i and b_i , respectively.

So, $\frac{dL}{dw_i}$ and $\frac{dL}{db_i}$ are, therefore, calculated in two passes: 1. Forward Pass 2. Backward Pass.

The forward pass is as discussed earlier in the feed forward neural network. During the forward pass, we keep track of all the layer outputs a_i and the input X .

In the backward pass, we perform series of calculations from the output layer to the first hidden layer. The derivatives for the $w_{k=\text{lastLayer}}$ and $b_{k=\text{lastLayer}}$ is the same as we discussed earlier. For derivatives w.r.t $w_{k \in \text{hiddenLayers}}$ and $b_{k \in \text{hiddenLayers}}$ are determined in the following method.

$$\frac{dL}{dw_{k \in \text{hiddenLayers}}} = \frac{dL}{da_k} \cdot \frac{da_k}{dz_k} \cdot \frac{dz_k}{dw_k} \quad (21)$$

From the previously discussed equations,

$$\frac{dz_{k \in \text{hiddenLayers}}}{dw_{k \in \text{hiddenLayers}}} = a_{k-1} \quad (22)$$

$$\frac{dL}{da_{k \in \text{hiddenLayers}}} = \frac{dL}{dz_{k+1}} \cdot w_{k+1} \quad (23)$$

Then, we can write $\frac{dL}{dw_{k \in \text{hiddenLayers}}}$ as,

$$\frac{dL}{dw_{k \in \text{hiddenLayers}}} = \frac{dL}{dz_{k+1}} \cdot w_{k+1} \cdot \frac{da_k}{dz_k} \cdot a_{k-1} \quad (24)$$

Therefore, we can similarly write,

$$\frac{dL}{db_{k \in \text{hiddenLayers}}} = \frac{dL}{dz_{k+1}} \cdot w_{k+1} \cdot \frac{da_k}{dz_k} \cdot 1 \quad (25)$$

From the above formulas, we can clearly visualize that the errors is propagating from the next layers to the current layer.

9 How are the weights updated?

Finally, the weights and biases in each gradient descent epoch is updated in the following manner.

$$w_i = w_i - \alpha \cdot \frac{dL}{dw_i} \quad (26)$$

$$b_i = b_i - \alpha \cdot \frac{dL}{db_i} \quad (27)$$

Here, α is the learning rate used to determine the scale of updating.

10 Code Implementations

Numpy-based implementation of a simple Neural Network can be found [here](#).

11 Resources

1. [Learning Backpropagation from Geoffrey Hinton](#)
2. [A beginner's guide to deriving and implementing backpropagation](#)
3. [Deep Learning by Professor Bryce](#)